

Entity Relationship Schema & Data Model (ERD)

Decoupled Graph-Relational Architecture — PostgreSQL Implementation

Version: 1.0.0

Date: July 2026

Status: Approved Schema

Engine: PostgreSQL 15+

1. Architecture Overview

The database architecture uses a **Decoupled Graph-Relational Model** in PostgreSQL. Personal metadata is stored independently in the **persons** table, while all direct genealogical connections are managed in a dedicated **relationships** junction table with status flags (**PENDING** vs **VERIFIED**).

2. Relational Schema DDL (PostgreSQL)

2.1 User Accounts Table (**users**)

```
CREATE TABLE users (  
  id BIGSERIAL PRIMARY KEY,  
  full_name VARCHAR(150) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  phone VARCHAR(50) UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  role VARCHAR(20) DEFAULT 'USER' CHECK (role IN ('USER', 'REVIEWER', 'ADMIN')),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

2.2 Family Individuals Table (**persons**)

```
CREATE TABLE persons (  
  id BIGSERIAL PRIMARY KEY,  
  first_name VARCHAR(100) NOT NULL,  
  father_name VARCHAR(100),  
  grand_father_name VARCHAR(100),  
  family_name VARCHAR(100),  
  gender VARCHAR(10) NOT NULL CHECK (gender IN ('MALE', 'FEMALE')),  
  is_alive BOOLEAN DEFAULT TRUE NOT NULL,  
  birth_date DATE,  
  birth_year INT,  
  death_date DATE,  
  burial_place VARCHAR(255),  
  photo_url TEXT,  
  biography TEXT,  
  created_by_user_id BIGINT REFERENCES users(id) ON DELETE SET NULL,  
  claimed_by_user_id BIGINT UNIQUE REFERENCES users(id) ON DELETE SET NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

2.3 Direct Relationships Table (`relationships`)

```
CREATE TABLE relationships (  
  id BIGSERIAL PRIMARY KEY,  
  person_id BIGINT NOT NULL REFERENCES persons(id) ON DELETE CASCADE,  
  related_person_id BIGINT NOT NULL REFERENCES persons(id) ON DELETE CASCADE,  
  relationship_type VARCHAR(20) NOT NULL CHECK (relationship_type IN ('PARENT', 'SPOUSE',  
'CHILD')),  
  status VARCHAR(20) DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'VERIFIED', 'REJECTED')),  
  created_by_user_id BIGINT REFERENCES users(id) ON DELETE SET NULL,  
  verified_by_user_id BIGINT REFERENCES users(id) ON DELETE SET NULL,  
  verified_at TIMESTAMP WITH TIME ZONE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT unique_relationship UNIQUE (person_id, related_person_id, relationship_type)  
);
```

2.4 Branch Reviewers Assignment Table (`branch_reviewers`)

```
CREATE TABLE branch_reviewers (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  root_person_id BIGINT NOT NULL REFERENCES persons(id) ON DELETE CASCADE,  
  assigned_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT unique_reviewer_branch UNIQUE (user_id, root_person_id)  
);
```

2.5 Deduplication & Merge Requests Table (`merge_requests`)

```
CREATE TABLE merge_requests (  
  id BIGSERIAL PRIMARY KEY,  
  primary_person_id BIGINT NOT NULL REFERENCES persons(id) ON DELETE CASCADE,  
  duplicate_person_id BIGINT NOT NULL REFERENCES persons(id) ON DELETE CASCADE,  
  status VARCHAR(20) DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'APPROVED', 'REJECTED')),  
  requested_by_user_id BIGINT REFERENCES users(id) ON DELETE SET NULL,  
  reviewed_by_user_id BIGINT REFERENCES users(id) ON DELETE SET NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

3. Entity Relationships & Cardinality

- **Users to Persons (`users` \$ ightarrow\$ `persons`):** One-to-Many (\$1 : N\$) for node creation (`created_by_user_id`), and unique One-to-One (\$1 : 1\$) for profile claiming (`claimed_by_user_id`).
- **Persons to Relationships (`persons` \$ ightarrow\$ `relationships`):** Many-to-Many (\$M : N\$) self-referential association managed via the junction table storing direct relation type and status.
- **Reviewers to Branches (`users` \$ ightarrow\$ `branch_reviewers` \$ ightarrow\$ `persons`):** Links a steward user to a root ancestor node, granting verification authority over all descendant sub-tree nodes.